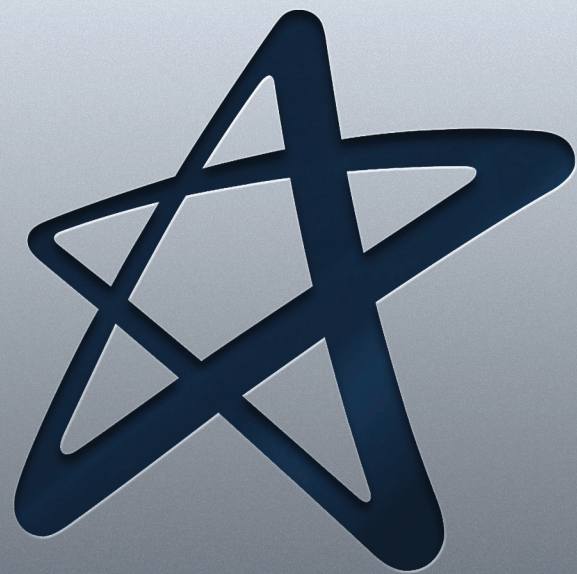


Compiladores e Interpretadores



Cruzeiro do Sul Virtual
Educação a distância

Material Teórico



Análise Léxica

Responsável pelo Conteúdo:

Prof. Dr. Cleber Silva Ferreira da Luz

Revisão Textual:

Prof.^a Esp. Kelciane da Rocha Campos

UNIDADE

Análise Léxica



- Introdução;
- Expressão Regular;
- Autômatos Finitos;
- *Lex*;
- Considerações Finais.



OBJETIVOS DE APRENDIZADO

- Apresentar todos os conceitos do analisador léxico;
- Demonstrar na prática como um analisador léxico é criado.



Orientações de estudo

Para que o conteúdo desta Disciplina seja bem aproveitado e haja maior aplicabilidade na sua formação acadêmica e atuação profissional, siga algumas recomendações básicas:



Assim:

- ✓ Organize seus estudos de maneira que passem a fazer parte da sua rotina. Por exemplo, você poderá determinar um dia e horário fixos como seu “momento do estudo”;
- ✓ Procure se alimentar e se hidratar quando for estudar; lembre-se de que uma alimentação saudável pode proporcionar melhor aproveitamento do estudo;
- ✓ No material de cada Unidade, há leituras indicadas e, entre elas, artigos científicos, livros, vídeos e *sites* para aprofundar os conhecimentos adquiridos ao longo da Unidade. Além disso, você também encontrará sugestões de conteúdo extra no item **Material Complementar**, que ampliarão sua interpretação e auxiliarão no pleno entendimento dos temas abordados;
- ✓ Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois irão auxiliar a verificar o quanto você absorveu de conhecimento, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e de aprendizagem.

Introdução

O processo de compilação é composto por duas fases: **análise** e **síntese**. Na fase de análise, o código-fonte é analisado para verificar se as instruções escritas no mesmo estão de acordo com a linguagem. A **análise léxica** é a primeira análise realizada na fase de análise de um compilador. O foco desta unidade é apresentar todos os conceitos e mecanismos envolvidos na análise léxica.

A análise léxica é realizada por um analisador léxico. Nesta análise, o código-fonte é visto como um arquivo de texto, onde o analisador léxico lê os caracteres e os separa em **elementos (Tokens)**. Cada elemento é formado por uma sequência finita de caracteres, que **representa uma unidade de informação do código-fonte**. Tais elementos podem ser, por exemplo, **palavras-chaves**, tais como **if** e **for**, **identificadores** e **símbolos especiais**, tais como, “<”, “>” e “^”, por exemplo (LOUDEN, 2004).

Podemos dizer que o analisador léxico **percorre** o código-fonte para **encontrar padrões** e que um **elemento** é uma **sequência de caracteres reconhecida por um padrão**. Por exemplo, os caracteres “f”, “o”, “r”, nesta ordem, casam com o padrão “**for**”, que é uma palavra-chave. Neste caso, o *for* é um *Token* de uma palavra reservada. O analisador léxico percorre o código-fonte a fim de encontrar situações semelhantes a essa, isto é, o analisador percorre o código-fonte identificando e separando os *Tokens*, que serão passados para a próxima análise, a análise sintática. É importante observar também que a fase de análise léxica é responsável por inicializar a tabela de símbolos. A Figura 1 ilustra um exemplo de processamento realizado por um analisador léxico. O analisador léxico obtém como entrada um conjunto de caracteres, neste caso uma instrução, e produz como saída um conjunto de *Tokens* que serão passados para o analisador sintático posteriormente. É importante lembrar que um *Token* é constituído por dois valores, o primeiro é a entidade que representa aquele *Token*, e o segundo é o valor daquela entidade. O segundo valor para o *Token* é opcional.

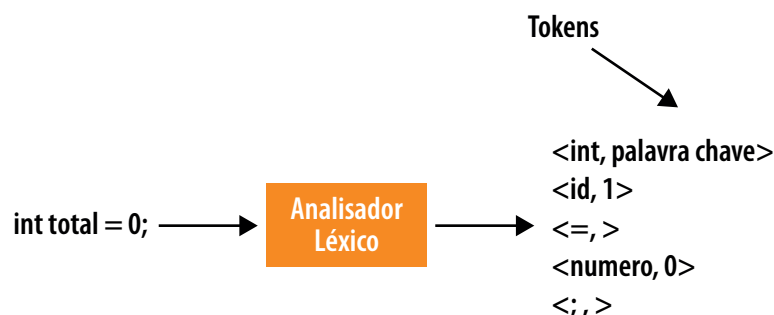


Figura 1 – Procedimento realizado por analisador léxico

A tarefa do analisador léxico é um caso especial de casamento de padrões, isto é, identificação de *Tokens*, certo? Dessa forma, um analisador léxico precisa utilizar métodos que permitem reconhecer e especificar padrões, tais métodos são basicamente

os de **expressões regulares** e de **autômatos finitos**. Expressões regulares e autômatos finitos são eficientes métodos para especificar e reconhecer padrões e ambos possuem o mesmo poder computacional. Ao implementar um analisador léxico, faz-se necessário o estudo detalhado sobre expressões regulares e autômatos finitos. Começaremos o estudo por expressões regulares e linguagens regulares.

Expressão Regular

Expressões Regulares representam padrões de **cadeias de caracteres**. Uma expressão regular r é completamente definida pelo conjunto de cadeias de caracteres com as quais ela casa. Expressão regular é definida a partir de conjuntos (linguagens) básicos e **operações** de **concatenação** e de **união**. As expressões regulares são indicadas para a comunicação humano x humano e, também, na comunicação humano x máquina (MENEZES, 2011).

Habitualmente, uma Expressão Regular é abreviada por **ER**. Uma Expressão Regular sobre um alfabeto Σ é indutivamente definida como se segue:

- **Base de Indução**

- » A expressão " $\emptyset$ " é expressão regular e denota a **linguagem vazia**:

$$\emptyset$$

- » A expressão " ϵ " é uma expressão regular e denota a linguagem que contém exclusivamente a **palavra vazia**:

$$\{ \epsilon \}$$

- » Para qualquer símbolo $x \in \Sigma$, a expressão " X " é uma expressão regular e denota a linguagem que contém exclusivamente a palavra constituída pelo símbolo x :

$$\{ X \}$$

- **Passo de Indução:** Se r e s são expressões regulares e denotam as linguagens R e S respectivamente, então:

- » **União:** A expressão " $(r+s)$ " é expressão regular e denota a linguagem:

$$R \cup S$$

- » **Concatenação:** A expressão " (rs) " é expressão regular e denota a linguagem:

$$R S = \{ uv \mid u \in R \text{ e } v \in S \}$$

- » **Concatenação Sucessiva:** A expressão " (r^*) " é expressão regular e denota a linguagem:

$$r^*$$

Se r é uma expressão regular, a linguagem correspondente é dita a **Linguagem Gerada** por r , sendo denotada por:

$$L(r) \text{ ou } \text{GERA}(R)$$

É possível omitir os parênteses em uma ER. Todavia, é necessário respeitar as seguintes convenções:

- A concatenação sucessiva tem precedência sobre a concatenação e a união;
- A concatenação tem precedência sobre a união.

Vale ressaltar que a operação de concatenação sucessiva também é chamada de **fecho de Kleene**.

Exemplo: Expressão regular

A seguir, é apresentada uma tabela com expressões regulares e as correspondentes linguagens geradas.

Tabela 1

Expressão Regular	Linguagem Gerada
Aa	somente a palavra aa
ba*	todas as palavras que se iniciam por b, seguido por zero ou mais a
(a+b)*	todas as palavras sobre {a, b}
(a+b)*aa(a+b)*	todas as palavras contendo aa como subpalavra
a*ba*ba*	todas as palavras contendo exatamente dois b
(a+b)*(aa+bb)	todas as palavras que terminam com aa e bb
(a+ε)(b+ba)*	todas as palavras que não possuem dois a consecutivos

Vamos analisar a linguagem gerada pela expressão $(a+b)^*(aa+bb)$:

a e b denotam $\{a\}$ e $\{b\}$, respectivamente.

$a + b$ denota $\{a\} \cup \{b\} = \{a,b\}$

$(a+b)^*$ denota $\{a,b\}^*$

aa e bb denotam $\{a\}a = \{aa\}$ e $\{b\}b = \{bb\}$, respectivamente.

$(aa+bb)$ denota $\{aa\} \cup \{bb\} = \{aa,bb\}$

$(a+b)^*(aa+bb)$ denota $\{a,b\}^*\{aa,bb\}$

Dessa forma, $\text{GERA}((a+b)^*(aa+bb))$ corresponde à seguinte linguagem: $\{aa,bb,abb,baa,aaaa,abaa,abbb,baaa,babb,bbba,bbbb,\dots\}$

Expressão regular é um poderoso mecanismo para descrever padrões de *Tokens*. Os *Tokens* de uma linguagem de programação tendem a se enquadrar em diversas **categorias** limitadas, que são padronizadas para as diferentes linguagens de programação. A categoria **Palavras Reservadas** são cadeias de caracteres alfabéticos com significado especial na linguagem. Por exemplo, na linguagem

JAVA temos *for*, *while*, *if*, entre outras. Podemos citar também a categoria de **Identificadores**, que usualmente são usados para identificar variáveis, funções, métodos, entre outros. Habitualmente, identificadores são definidos como sequências de letras e dígitos iniciando por uma letra. Outras categorias são a dos **constantes**, que podem ser constantes numéricas, como 55 e 3.144, e dos **literais**, que são cadeias de caracteres como “olá, mundo” e caracteres como “a” e “b” (LOUDEN, 2004). Vamos descrever expressões regulares típicas para algumas categorias.

- **Palavras reservadas e identificadores:** palavras reservadas são as mais simples de escrever como expressões regulares: elas são representadas por sequências fixas de caracteres. **Se quisermos juntar todas as palavras reservadas em uma única definição, podemos escrever algo como**

$$\text{reservadas} = \text{if} \mid \text{while} \mid \text{else} \dots$$

Todavia, **identificadores** são cadeias de caracteres que não são fixas. Frequentemente, um identificador deve iniciar com uma letra e conter somente letras e dígitos. Podemos expressar isso em termos de definições regulares:

$$\text{letra} = [\text{a-zA-Z}]$$
$$\text{dígito} = [0-9]$$
$$\text{identificador} = \text{letra}(\text{letra} \mid \text{dígito})^*$$

Podemos expressar **números naturais** da seguinte forma:

$$\text{natural} = [0-9]^+$$
$$\text{signedNatural} = (+ \mid -) ? \text{natural}$$

O “?” indica que o “+” ou “-” é opcional.

- **Ambiguidade:** habitualmente, na descrição de *Tokens* em linguagem de programação utilizando expressões regulares, algumas cadeias de caracteres podem casar com diversas expressões regulares. Por exemplo, cadeias como *for* e *else* poderiam ser **identificadores** ou **palavras reservadas**. Semelhante aos elementos *<>*, que poderiam ser interpretados como representação de elementos (“**menor**” e “**maior**”) ou um único elemento (“**diferente**”). Assim, uma definição de linguagem de programação deve determinar que interpretação deve ser observada, e as expressões regulares não podem fazer isso. O projetista da linguagem de programação em desenvolvimento deve definir **regras de eliminação de ambiguidade**, que indicam o significado para cada caso (LOUDEN, 2004).

Para os exemplos citados acima, há duas regras típicas. Na primeira regra, quando uma cadeia de caracteres pode ser um **identificador** ou uma **palavra-chave**, a interpretação de **palavra-chave** deve prevalecer. Já a segunda diz que quando uma cadeia de caracteres pode ser um **único elemento** ou uma **sequência de elementos**, a interpretação de elemento **único elemento** deve prevalecer.

Autômatos Finitos

Um Autômato é um formalismo matemático reconhecedor de linguagens. Sendo composto por estados e transações, um Autômato reconhece se uma linguagem pertence a um alfabeto. Autômatos finitos podem ser utilizados para descrever o processo de reconhecimento de padrões em cadeias de entrada e, assim, podem ser utilizados para construir analisadores léxicos. Vamos lembrar rapidamente o que é um Autômato Finito e quais são os tipos de Autômatos existentes.

Um autômato pode ser considerado um sistema de estados, onde dada uma entrada o sistema assume um estado ou um conjunto de estados bem definidos. Formalmente, um Autômato Finito Determinístico (AFD) é uma 5-upla ordenada:

$$M = (\Sigma, Q, \delta, q_0, F)$$

onde,

- Σ é o alfabeto de entrada;
- Q é o conjunto finito de estados possíveis do autômato;
- δ é uma função programa, também chamada de função transição. Por exemplo, vamos supor que a função programa é definida para um estado p e um símbolo a , resultando no estado q , então temos:

$$\delta(p,a) = q$$

A função programa é responsável pela transição de estados nos autômatos. No exemplo acima, o sistema estava no estado p ; ao ler um símbolo a , ele passa para o estado q ;

- q_0 é um elemento distinguindo de Q , chamado de estado inicial;
- F é um subconjunto de Q , chamado de estados finais.

Um Autômato pode ser representado através de diagrama. Nesta representação, estados são representados por círculos. As transações são representadas por setas que indicam a direção da transição, conforme ilustrado na Figura 2(a). O estado inicial é diferenciado por uma seta que faz menção por qual estado o processamento deve começar. Já o estado final é representado por uma borda. O estado inicial e o estado final são ilustrados na Figura 2(d). Transições paralelas também são aceitas e são representas por setas paralelas (Figura 2(b)), ou simplesmente por uma seta, mas com dois símbolos em cima (Figura 2(c)).

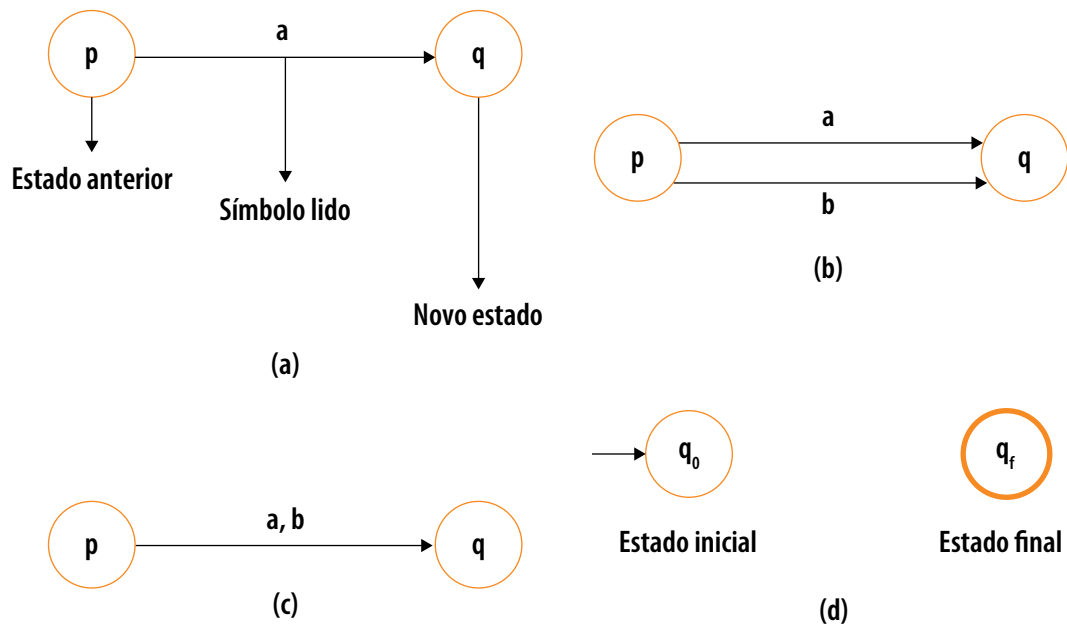


Figura 2 – Exemplos de autômato finito determinístico

Fonte: Adaptado de Menezes, 2011

Um autômato pode ser:

- **Determinístico:** dependendo do símbolo lido e do estado corrente (atual), o sistema pode assumir um único estado bem definido;
- **Não determinístico:** dependendo do símbolo lido e do estado corrente (atual), o sistema pode assumir um conjunto de estados alternativos;
- **Com movimento vazio:** dependendo do estado atual e sem ler nenhum símbolo, o sistema pode assumir um conjunto de estados.

Em **autômatos determinísticos**, como o próprio nome induz, a transição de um estado para o outro é determinístico, isto é, a transição é certa. Em contrapartida, em **autômatos não determinísticos**, a transição pode assumir um conjunto de estados, isto é, a transição não é certa. Dado um conjunto de estados possíveis, o sistema pode assumir qualquer um. Em **autômatos com movimento vazio**, sem ler nenhum símbolo, o sistema pode assumir um **conjunto de estados possível**. Um autômato com movimentos vazios também é não determinístico. Autômatos com movimentos vazios podem ser entendidos como movimentos encapsulados, nas quais, excetuando-se por uma eventual mudança de estado, nada mais pode ser observado; de forma análoga, podemos citar o encapsulamento das linguagens orientadas a objetos.

Em termos computacionais, os três tipos de autômatos possuem o mesmo poder computacional. Autômatos são sistemas de estados finitos. Assim, autômatos determinísticos também são chamados de autômatos finitos determinísticos, ou simplesmente de **AFD**. Autômatos não determinísticos também são chamados de autômatos finitos não determinísticos, ou simplesmente de **AFN** (MENEZES, P. B, 2011).

Simulação em autômatos

Autômatos são mecanismos para reconhecimento de padrões. Na análise léxica, autômatos são usados para reconhecer padrões de caracteres que formam *Tokens*. Por exemplo, a palavra *if* é composta pelos caracteres “i” e “f”, nesta disposição esses caracteres casam com o padrão “*if*”, palavra reservada. Agora, como utilizar autômatos para realizar tal reconhecimento? A resposta é simples, é criado um autômato que consiga reconhecer tal padrão. Por exemplo, a Figura 3 apresenta um autômato para o reconhecimento da palavra reserva *if*.

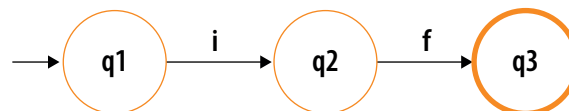


Figura 3 – Autômato para reconhecer o padrão *if*

É impossível realizar uma rápida simulação e constatar que esse autômato é capaz de reconhecer todas as palavras “*if*”. Dada a palavra “*if*”, observe que o processamento começa no estado q1, que é o estado inicial, e após ler um “i”, avançamos para o estado q2, e ao ler um “f”, avançamos para o estado q3, que é um estado final. O processamento termina em um estado final; assim, a sequência de caracteres “i” “f” corresponde ao padrão “*if*”.

Das Expressões Regulares para Autômatos Finitos Determinísticos

Há uma forte relação entre autômatos finitos (AFD) e Expressões Regulares (ER). Nesta seção, é apresentado um algoritmo para traduzir uma expressão regular em autômato finito determinístico. As noções de AFD para ER e de ER para AFD são equivalentes (LOUDEN, 2004).

O algoritmo que traduz expressão regular para um DFA usa uma construção intermediária, em que um autômato finito não determinístico (AFN) é derivado da expressão regular, posteriormente o AFN é utilizado para construir o DFA equivalente. Há algoritmos que traduzem as expressões regulares para DFA, todavia estes algoritmos são muito complexos. O algoritmo que será descrito a seguir, que utiliza uma construção intermediária, é bem mais simples.

Serão descritos dois algoritmos para traduzir uma ER em AFN e outro para traduzir um AFN em um AFD. O primeiro algoritmo que será descrito será para traduzir o ER em AFN.

De uma expressão regular para um AFN

A construção que será descrita nesta seção é conhecida como a **construção de Thompson**. Essa construção utiliza ϵ -transições (movimentos vazios) para juntar cada pedaço de uma expressão regular e formar um AFN correspondente à expressão toda. Assim, será exibido um AFN para cada expressão regular básica e,

em seguida, será apresentado como cada operação de expressão regular pode ser obtida pela conexão de AFNs das subexpressões (LOUDEN, 2004).

- **Expressões regulares básicas:** uma expressão regular básica possui a forma a e ϵ , em que a representa um casamento de um único caractere do alfabeto, ϵ representa um casamento com a cadeia vazia. Um AFN equivalente à expressão regular a é:

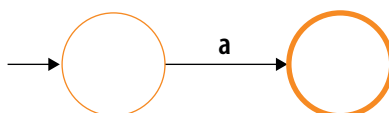


Figura 4 – Um AFN equivalente à expressão regular a

Fonte: Adaptado de LOUDEN, 2004

De maneira semelhante, um AFN equivalente ϵ é:

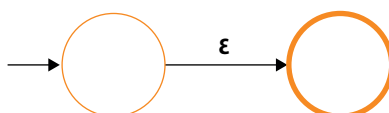


Figura 5 – Um AFN equivalente à expressão regular ϵ

Fonte: Adaptado de LOUDEN, 2004

- **Concatenação:** em expressões regulares, a concatenação de uma expressão r com a expressão s resulta em rs . Agora, vamos criar um AFN equivalente a essa concatenação. Vamos assumir que AFNs equivalentes a r e s já tenham sido construídos. Podemos expressar isso da seguinte forma, para o AFN corresponde a r , e de maneira similar para s :

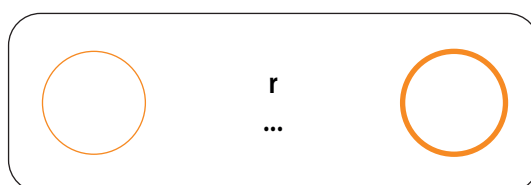


Figura 6 – AFN corresponde a r

Fonte: Adaptado de LOUDEN, 2004

No diagrama acima, as reticências indicam os estados e transições dentro do AFN, que, por simplicidade, não são mostrados. Neste diagrama, assumimos que o AFN correspondente a r tem somente um estado de aceitação (estado final) (LOUDEN, 2004). Para um AFN correspondente a rs , temos:

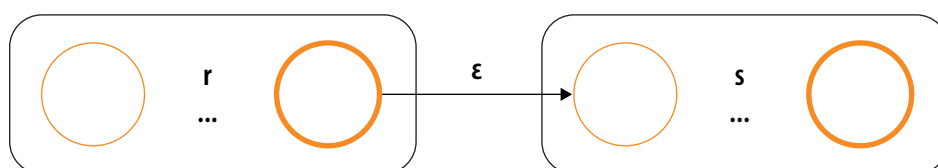
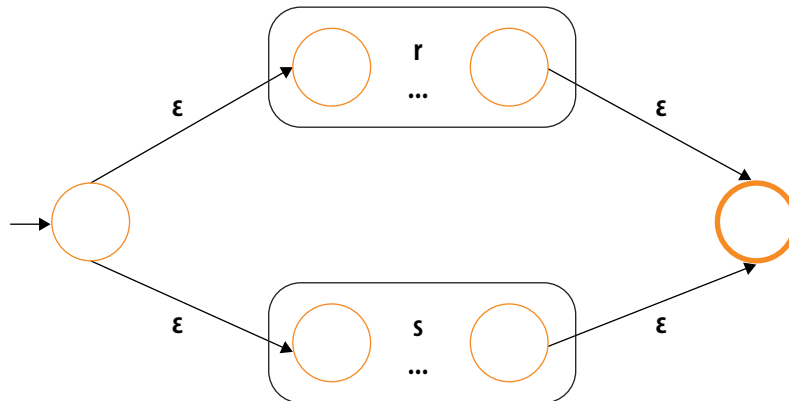


Figura 7 – AFN corresponde a rs

Fonte: Adaptado de LOUDEN, 2004

Conectamos o estado final de r com o estado inicial de s por uma transição ϵ -transição. O novo AFN possui o estado inicial de r e o estado final de s .

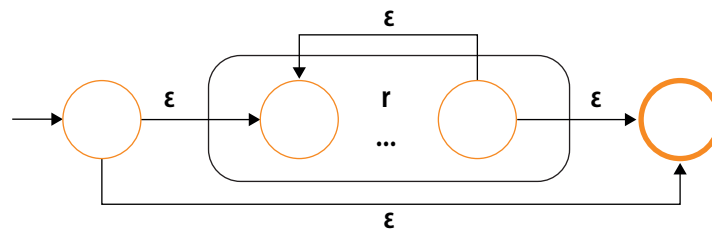
- **Escolha entre alternativas:** queremos construir um AFN que corresponda a $r|s$ com base nas mesmas hipóteses. Isso é realizado da seguinte maneira:

Figura 8 – AFN corresponde a $r|s$

Fonte: Adaptado de LOUDEN, 2004

Neste caso, adicionamos um novo estado inicial e um novo estado final e os conectamos com ϵ -transições (movimentos vazios).

- **Concatenação sucessiva (repetição):** agora, queremos construir um AFN que corresponda a r^* , dada um AFN que corresponda a r . Fazemos o seguinte:

Figura 9 – AFN corresponde a r^*

Fonte: Adaptado de LOUDEN, 2004

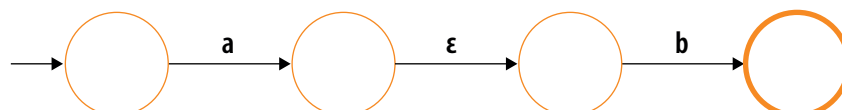
Também neste caso, acrescentamos um estado inicial e um estado final. Neste AFN, a repetição advém da nova ϵ -transição em r .

Exemplo: traduzir a expressão **abla** em um AFN segundo a construção de Thompson. Primeiro, formamos os autômatos para as expressões regulares básicas **a** e **b**.

Figura 10 – AFNs para **a** e **b**

Fonte: Adaptado de LOUDEN, 2004

Agora, formamos o AFN para concatenação **ab**:

Figura 11 – AFN para **ab**

Fonte: Adaptado de LOUDEN, 2004

Por fim, faremos outra cópia do AFN de **a** e utilizamos na construção do AFN de **abla**a.

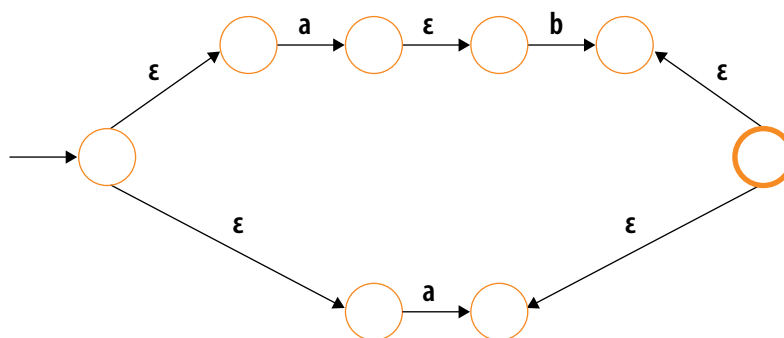


Figura 12 – AFN para **ab|a**
Fonte: Adaptado de LOUDEN, 2004

De um AFN para AFD

Agora, iremos descrever um algoritmo que, dado um AFN qualquer, construirá um AFD equivalente (isto é, um AFD que aceite precisamente as mesmas cadeias que o AFN aceita). O primeiro passo é criar um método para eliminar as ϵ -transições e as transições múltiplas a partir de um estado para um caractere de entrada único. Segundo Louden (2004), a **eliminação das ϵ -transições requer a construção de ϵ -fechos**. Um ϵ -fecho pode ser considerado como um conjunto de todos os estados atingíveis por ϵ -transições a partir de um estado ou um conjunto de estados. A eliminação de transições multiplicas a partir de um estado para um caractere, requer o acompanhamento do conjunto de estados atingíveis pelo casamento com um único caractere. Os dois processos nos levam a considerar conjuntos de estados em vez de estados únicos. Dessa forma, não é de se surpreender que o AFD construído tenha como seus estados conjuntos de estados do AFN original. Dessa forma, o algoritmo é denominado **construção de subconjuntos**. Primeiro, vamos discutir o ϵ -fecho com mais detalhes e, então, seguimos com uma descrição da construção de subconjuntos.

- **O ϵ -fecho de um conjunto de estados:** definimos o ϵ -fecho de um único estado s como sendo o conjunto de estados atingíveis por uma série com zero ou mais ϵ -transições e denotamos esse conjunto como $\bar{s}$. É importante observar que ϵ -fecho de um estado sempre contém o próprio estado (LOUDEN, 2004).

Exemplo: o AFN a seguir corresponde à expressão regular a^* pela construção de Thompson.

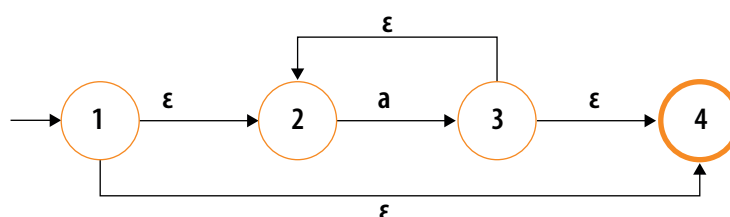


Figura 13 – AFN para a^*
Fonte: Adaptado de LOUDEN, 2004

Neste AFN temos $\bar{1} = \{1, 2, 4\}$, $\bar{2} = \{2, 3\}$, $\bar{3} = \{2, 3, 4\}$ e $\bar{4} = \{4\}$

Vamos definir, agora, o ε -fecho de um conjunto de estados como a união dos ε -fecho de cada estado individual. Considerando o AFN acima, temos $\{1, 3\} = \bar{1} \cup \bar{3} = \{1, 2, 4\} \cup \{2, 3, 4\} = \{1, 2, 3, 4\}$.

- **Construção de subconjuntos:** Agora, vamos descrever o algoritmo para a construção de AFD a partir de um AFN $\bar{M}$. Neste algoritmo, primeiro computamos o ε -fecho do estado inicial de M , esse passa a ser o estado inicial de $\bar{M}$. Para esse conjunto, e para cada conjunto subsequente, computamos transições de caracteres a da seguinte forma. Dado um conjunto S de estados e um caractere a do alfabeto, computamos o conjunto $S'_a = \{t \mid \text{para algum } s \text{ em } S \text{ existe uma transição de } s \text{ para } t \text{ em } a\}$. Portanto, computamos $\bar{S}'_a$, o ε -fecho de S'_a . Assim define um novo estado na construção de subconjuntos juntamente com uma nova transição $S \xrightarrow{a} \bar{S}'_a$. Esse processo continua até que novos estados e transições não sejam mais criados. Os estados finais serão aqueles que contenham um estado final do AFN (LOUDEN, 2004).

Agora, consideremos o AFN apresentado como exemplo nesta seção. O estado inicial do AFD correspondente é $\bar{1} = \{1, 2, 4\}$. Há uma transição do estado 2 para o 3 em a , e nenhuma transição dos estados 1 ou 4 em a ; assim, há uma transição em a de $\{1, 2, 4\}$ para $\{\bar{1}, 2, 4\}_a = \{\bar{3}\} = \{2, 3, 4\}$. Como não há outras transições em um caractere de qualquer um dos estados 1, 2 e 4, passamos para o novo estado $\{2, 3, 4\}$. Há uma transição de 2 para 3 em a e nenhuma a -transição de 3 ou 4; assim, há uma transição de $\{2, 3, 4\}$ para $\{2, 3, 4\}_a = \{\bar{3}\} = \{2, 3, 4\}$. Logo, há uma a -transição de $\{2, 3, 4\}$ para si próprio. Agora, resta apenas observar que o estado 4 do AFN é o estado final e como tanto $\{1, 2, 4\}$ quanto $\{2, 3, 4\}$ contêm 4, ambos são estados final do AFD correspondente. A seguir, é apresentado o diagrama do AFD construído.

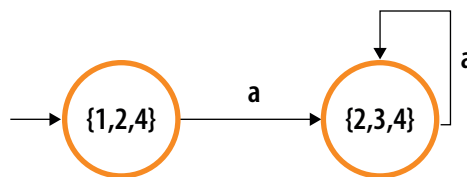


Figura 14 – AFD

Fonte: Adaptado de LOUDEN, 2004

Linguagem Regular

Uma linguagem L é dita uma linguagem regular se existe pelo menos um Autômato Finito Determinístico que aceita L .

Exemplo: Autômato Finito: Número par de cada símbolo

Considere a seguinte linguagem sobre o alfabeto $\{a, b\}$:

$$L_4 = \{w \mid w \text{ possui um número par de } a \text{ e um número par de } b\}$$

O autômato finito:

$$M_4 = (\{a, b\}, \{q_0, q_1, q_2, q_3\}, \delta_4, q_0, \{q_0\})$$

Este Autômato é ilustrado na Figura 7. Este Autômato é tal que aceita $(M_4) = L_4$. Dessa forma, L_4 é uma linguagem regular.

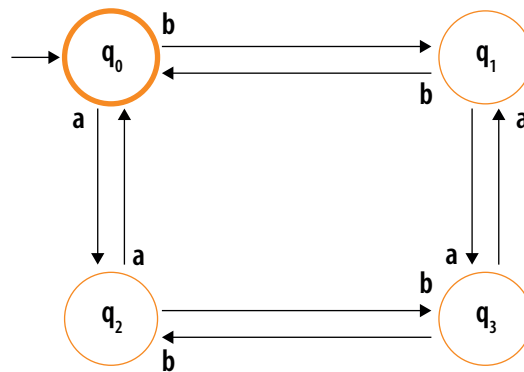


Figura 15 – Diagrama AFD: número par de cada símbolo
Fonte: Adaptado de MENEZES, 2011

Lex

Proposto por Eric Schmidt e Mike Lesk, o *Lex* é um programa que permite gerar analisadores léxicos. Há diversas versões do *Lex*, a mais popular é a ***flex***, na qual há uma abstração para *JAVA* chamada de ***JFlex***. Nesta disciplina, iremos trabalhar com o ***JFlex***. *Lex* é um programa que recebe como entrada um arquivo de texto contendo expressões regulares, juntamente com as ações associadas a cada expressão. O ***JFlex*** produz um arquivo de saída contendo código *JAVA* que define um procedimento ***yylex*** – uma implementação baseada em tabelas de um AFD que corresponde às expressões regulares do arquivo de entrada e que opera como um procedimento ***getToken***. O arquivo de saída do ***JFlex*** geralmente denominado *lex.yy*

Segundo LOUDEN, 2004, o *Lex* permite o casamento de caracteres únicos, ou de cadeias de caracteres. O *lex* interpreta os símbolos *, +, (,) e | de maneira usual e também utiliza o ponto de interrogação (?) como símbolo para indicar um item opcional. Como exemplo de notação do *Lex*, podemos escrever uma expressão regular para o conjunto de caracteres de a's e b's iniciando com aa ou com bb com um c opcional no fim.

$$(aa|bb)(a|b)^*c?$$

Ou ainda:

$$("aa"|"bb")("a"|"b")^*"c"?$$

A convenção *Lex* para classes de caracteres é escrever entre colchetes. Por exemplo, `[abxz]` indica qualquer um dos caracteres `a`, `b`, `x` e `z`. Assim, é possível escrever a expressão acima da seguinte forma:

$$(aa|bb)[ab]^*c?$$

Intervalos de caracteres também podem ser escritos da seguinte forma: `[0-9]`, isso significa um intervalo entre os números de 0 a 9.

Um arquivo de entrada *Lex* é composto por três partes: a primeira parte corresponde a um conjunto de **definições**, a segunda a um conjunto de **regras**, e a última corresponde a um conjunto de **rotinas de usuários**.

{definições}

%%

{regras}

%%

{rotinas auxiliares}

Considerações Finais

A análise léxica é a primeira análise realizada no processo de compilação. Nesta análise, o código-fonte é visto como um arquivo texto, onde o analisador léxico lê os caracteres e os separa em *Tokens*. Um **Token** pode ser considerado como uma unidade de informação do código-fonte. São exemplos de *Tokens*: palavras-chaves, identificadores, números, entre outros. O objetivo do analisador léxico é ler o código-fonte e encontrar padrões no texto. Quando uma sequência de caracteres casa com um padrão, temos um *Token*.

Habitualmente, expressões regulares e autômatos são utilizados na criação de um analisador léxico. Alguns compiladores utilizam as expressões regulares para descrever os padrões de *Token* e utilizam os autômatos, para identificar padrões.

Expressões Regulares representam padrões de **cadeias de caracteres**. Uma expressão regular r é completamente definida pelo conjunto de cadeias de caracteres com as quais ela casa. Expressão regular é definida a partir de conjuntos (linguagens) básicos e **operações** de **concatenação** e de **união**. Expressão regular é um poderoso mecanismo para descrever padrões de *Tokens*. Os *Tokens* são organizados em categorias. Existem diversas categorias de *Token*, podemos citar como exemplo a categoria das **Palavras Reservadas**, que são, por exemplo, *for*, *while* e *if*. Outra categoria de **Identificadores**, que usualmente são usados para identificar variáveis, funções, métodos, entre outros.

Um Autômato é um mecanismo poderoso de reconhecimento de padrões. Existem três tipos de autômatos. O autômato finito determinístico lê um símbolo e

assume um único estado. O autômato finito não determinístico lê um símbolo e assume um conjunto de estado. Já o autômato com movimento vazio sem ler nenhum símbolo pode assumir um conjunto de estados.

Uma forma de construir um analisador léxico é através do *Lex*. O *Lex* é um programa que permite gerar analisadores léxicos. Há diversas versões do *Lex*, a mais popular é a ***flex***, na qual há uma abstração para *JAVA* chamada de ***JFlex***. *Lex* é um programa que recebe como entrada um arquivo de texto contendo expressões regulares, juntamente com as ações associadas a cada expressão.

O estudo de análise léxica é muito importante para um profissional de computação. Com o entendimento correto desta análise, é possível compreender de forma clara a importância de um analisador léxico em um compilador. Também é possível utilizar o conhecimento adquirido no estudo desta análise em outros campos, tais como a robótica. É possível implementar um sistema de leitura de texto de um robô utilizando analisadores léxicos.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:



Livros

Construindo compiladores

COOPER, K. D.; TOREZON, L. **Construindo compiladores**. 2ª edição. São Paulo: Elsevier, 2014.



Leitura

Basics of Compiler Design

MOGENSEN, T. A. **Basics of Compiler Design**. Department of Computer Science. University of Copenhagen, 2010.

<http://bit.ly/2LorUn2>

Compiler design: theory, tools, and examples

BERGMANN, S. D. **Compiler design: theory, tools, and examples**. 2016.

<http://bit.ly/2LjFEPO>

Compiler construction

WAITE, W. M.; GOOS, G. **Compiler construction**. 1996.

<http://bit.ly/2LoT2Cu>

Referências

AHO, A. V. **Compiladores**: princípios, técnicas e ferramentas. 2ª ed. São Paulo: Pearson, 2008. (E-book)

APPEL, A. W. **Modern compiler implementation in Java**. 2. ed. New York: Cambridge University Press, 2002.

LOUDEN, K. C. **Compiladores**: princípios e práticas. São Paulo: Pioneira Thomson Learning, 2004.

MENEZES, P. B. **Linguagens formais e autômatos**. 6ª ed. São Paulo: Bookman, 2011.

PRICE, A. M. A. **Implementação de linguagens de programação**: compiladores. 2ª ed. Porto Alegre: Sagra Luzzatto, 2001.

RAMOS, M. V. M.; VEGA, I. S.; JOSÉ NETO, J. **Linguagens formais**: teoria, modelagem e implementação. Porto Alegre: Bookman, 2009. 656 p.

WAITE, W. M.; GOOS, G. **Ompiler Construction**. Disponível em: <<http://www.cs.cmu.edu/~aplatzer/course/Compilers/waitegoos.pdf>>. Acesso em: 20/08/19. (E-book)



Cruzeiro do Sul
Educatonal